INSA INSTITUT NATIONAL DES SCIENCES APPLIQUÉES ROUEN

Projet Scientifique Encadré PSE
STPI/PSE/2025-03

Création de sites web interactifs : illustration du principe des échanges client/serveur



Étudiants :
Maïssa BEN SETHOUM
Bréhan ERAUD-ROBERT
Wallis BOULLY
Mohammed KHOUFI
Zayd BENLGHAZI
Corentin BLANQUET

Enseignant-responsable du projet :
Alexis LECOMTE

Date de remise du rapport : 13/06/2025

Référence du projet : STPI/PSE/2025 – 03

Intitulé du projet : Création de sites web interactifs : illustration du principe des échanges client/serveur

Type de projet : Numérique

Objectifs du projet : Ce projet a pour but de comprendre et d'appliquer le fonctionnement des échanges client/serveur à travers la création d'un site web interactif. Il permet de se familiariser avec l'utilisation des API, devenues incontournables dans le développement web moderne, ainsi qu'avec la structuration d'un projet informatique complet. L'objectif est également de renforcer les compétences en travail collaboratif en groupe, en favorisant la répartition des tâches et la coordination entre les membres. Le projet enseigne aussi comment intégrer efficacement des données externes dans une interface à la fois fonctionnelle et personnalisée. Enfin, il constitue une première approche concrète du développement web dynamique, en mêlant logique, design et interactivité.

Mots-clefs du projet : API, client-serveur, site web, site statique

Table des matières

Notations **4**

Introduction **5**

1 Méthodologie, organisation du travail **6**
1.1 Compréhension du sujet et répartition initiale 6
1.2 Recherche, brainstorming et organisation technique 6
1.3 Collaboration et finalisation du projet 7

2 Travail réalisé et résultats **8**
2.1 Partie Front-End 8
2.1.1 Réalisation de la page d'accueil 8
2.1.2 Réalisation de la page de sélection 10
2.1.3 Réalisation de la page principale 12
2.2 Partie Logique 15
2.2.1 Recherche des APIs 15
2.2.2 Utilisation des APIs 15
2.2.3 Interface unifiée pour l'utilisation des APIs 16
2.2.4 Récupération des choix utilisateurs 18
2.2.5 Intéractions avec le HTML 20
2.2.6 Améliorations 22

Conclusion et perspectives **24**

A Notre usage de l'IA dans ce projet **25**

B Liens de documentation **26**

C Code Supplémentaire **27**

Notations et Acronymes

API : Application Programming Interface (interface de programmation d'application)

HTML : HyperText Markup Language. Langage de balisage utilisé pour structurer le contenu d'une page web (titres, paragraphes, images...).

CSS : Cascading Style Sheets. Langage de style utilisé pour définir l'apparence d'une page HTML (couleurs, polices, marges...).

Front-end : Partie visible d'un site web avec laquelle l'utilisateur interagit directement. Il inclut le HTML, le CSS et le JavaScript.

Back-end : Partie invisible d'un site, côté serveur. Il gère la logique, la base de données, et les échanges avec le front-end.

JSON : JavaScript Object Notation. Format léger d'échange de données entre client et serveur, facile à lire et à manipuler.

Promise : Objet JavaScript représentant le résultat futur d'une opération asynchrone, elle prend en charge la réussite comme l'échec de la fonction.

URL : Uniform Resource Locator. Adresse permettant d'accéder à une ressource sur le web.

HTTPS : HyperText Transfer Protocol Secure. Version sécurisée du protocole HTTP, utilisant le chiffrement SSL/TLS.

DOM : Document Object Model. Représentation en mémoire de la structure d'un document HTML, manipulable en JavaScript.

UI : User Interface. Ensemble des éléments graphiques permettant à l'utilisateur d'interagir avec un site ou une application.

Introduction

Les technologies du web ne cessent d'évoluer et jouent un rôle central dans la manière dont les informations sont diffusées, partagées et personnalisées. Au cœur de cette transformation se trouvent les API (Application Programming Interface), qui permettent à différentes applications de communiquer entre elles, de partager des données en temps réel et d'enrichir les services proposés aux utilisateurs.

Historiquement, les premières formes d'API sont apparues dans les années 2000 avec l'essor des services web. Leur généralisation a radicalement changé la manière de concevoir les applications modernes. Grâce à elles, il est désormais possible de créer des plateformes connectées, dynamiques et intelligentes, en tirant parti des données mises à disposition par d'autres services (météo, actualités, musique, géolocalisation, etc...). Ce mode d'échange client/serveur est aujourd'hui au cœur du fonctionnement d'Internet.

Dans le cadre de ce projet, nous sommes amenés à découvrir les API, à comprendre leur fonctionnement, et surtout à les mettre en application à travers la création d'un site web interactif. Ce travail s'inscrit dans un objectif pédagogique : nous initier à la logique des échanges client/serveur, à la structuration d'un projet en groupe, et à la mise en œuvre concrète d'un développement web intégrant des ressources externes.

Le projet s'est déroulé sur 13 semaines, encadré par notre enseignant, et nous a permis de travailler en groupe, de nous répartir les tâches, et de confronter nos idées afin de proposer un site fonctionnel exploitant des APIs de manière pertinente. Ce rapport retrace l'ensemble de notre démarche, de la méthodologie adoptée jusqu'au résultat obtenu, en passant par les difficultés rencontrées, les choix techniques réalisés et les enseignements tirés de cette expérience.

Le site web que nous avons réalisé et que nous allons présenter dans ce rapport est accessible au lien suivant. De même, le code vers notre projet Git est disponible à ce lien. (Pour le rapport écrit, les liens sont disponibles dans l'annexe des liens).

Chapitre 1

Méthodologie, organisation du travail

1.1 Compréhension du sujet et répartition initiale

Le projet, intitulé "Création de sites web interactifs : illustration du principe des échanges client/serveur", visait à concevoir un site web tout en illustrant l'utilisation des API dans les échanges entre client et serveur. Il s'est déroulé sur 13 semaines, avec un suivi hebdomadaire permettant de faire le point sur l'avancement et d'ajuster la direction.

Mr Lecomte a d'abord organisé les membres du groupe en deux sous-groupes distincts. Le premier groupe (Mohammed, Corentin et Zayd) se chargeait de la recherche sur les API : leur fonctionnement, utilité, et intégration dans un site web. Le deuxième groupe (Maïssa, Bréhan et Wallis) était chargé de définir le concept du site et de déterminer ce qu'il contiendrait.

Initialement, nous avons proposé plusieurs idées de projets, tels qu'un site répertoriant les projets informatiques des STPI 2, mais le professeur a rapidement orienté notre réflexion vers l'utilisation des API pour illustrer les échanges client/serveur. Nous avons exploré plusieurs pistes, comme un site météo, un site musical, un site de suggestions de films, et un site d'actualités personnalisées. Cette dernière option, qui permettrait aux utilisateurs de personnaliser leur contenu en fonction de leurs préférences (politique, sport, science, etc.), a été retenue pour sa faisabilité technique et son adéquation avec le thème du projet.

1.2 Recherche, brainstorming et organisation technique

Au début du projet, chaque groupe approfondissait ses recherches. Le premier groupe se concentrait sur la compréhension des API et leur disponibilité, découvrant de nombreuses plateformes comme YouTube, Spotify, ou des bases de données météo et d'actualités. L'idée principale était de créer un site personnalisé, en recommandant du contenu en fonction des préférences des utilisateurs. Cependant, cette idée s'est révélée trop complexe, notamment en raison de l'ajout d'un système d'amis, rapidement jugé irréaliste.

Bréhan a aidé à évaluer la faisabilité des idées et a mis en lumière les limitations techniques, telles que les coûts d'utilisation des API ou les limitations des requêtes, ce qui a poussé le groupe à se concentrer sur un projet plus simple. Après quelques discussions, le groupe a retenu un site d'actualités personnalisées, utilisant des API gratuites telles que NewsAPI. Ce projet offrait une personnalisation utilisateur et était réalisable avec les

ressources disponibles.

Une fois le projet choisi, le travail a été réparti en trois sous-groupes : Mohammed et Maïssa ont travaillé sur la structure du site et la partie front-end (HTML/CSS). Wallis, Bréhan et Corentin ont été responsables de la recherche et de l'intégration des API de news, en testant leur fiabilité et leurs limitations. Zayd s'est formé à JavaScript pour comprendre comment envoyer des requêtes API et intégrer les données dans le site.

Chaque sous-groupe a partagé ses avancées, souvent via des échanges sur un groupe Instagram pour une communication rapide et via un Google Docs pour centraliser les informations et les progrès du projet. GitLab a été utilisé pour gérer le code et suivre l'évolution du projet.

1.3 Collaboration et finalisation du projet

La communication au sein du groupe s'est également organisée autour des points réguliers avec le professeur. Chaque séance commençait par un bilan rapide permettant de poser des questions, de soulever d'éventuels problèmes, et de réajuster la répartition des tâches. Le professeur agissait en tant que coordinateur, sans chef de groupe désigné, afin de favoriser une dynamique collaborative.

Avant les vacances, l'objectif était d'avoir une version fonctionnelle du site. Chaque sous-groupe a donc avancé sur ses tâches : Mohammed et Maïssa ont finalisé la structure des trois pages principales, Bréhan a intégré du JavaScript pour rendre les boutons fonctionnels et permettre la transmission des informations entre les pages, Wallis et Corentin ont intégré le code permettant d'afficher dynamiquement les actualités à partir des API. Zayd s'est chargé de rendre la disposition des actualités aléatoire

À la rentrée, la répartition des tâches a été affinée pour finaliser le projet. Maïssa et Mohammed ont commencé à rédiger le rapport, Bréhan et Zayd ont poursuivi l'intégration du JavaScript et la mise en ligne du site, et Wallis et Corentin ont résolu les problèmes d'affichage, notamment la duplication des actualités provenant de différentes API et des images manquantes.

Cette organisation en sous-groupes a permis à chacun de progresser techniquement et de trouver sa place dans le projet, tout en maintenant une cohésion générale grâce à une coordination régulière. Elle a aussi facilité l'adaptation aux défis techniques rencontrés tout au long du projet.

Chapitre 2

Travail réalisé et résultats

2.1 Partie Front-End

2.1.1 Réalisation de la page d'accueil

La création de l'interface utilisateur de notre site repose essentiellement sur deux langages web fondamentaux : HTML (HyperText Markup Language) et CSS (Cascading Style Sheets). Le HTML est utilisé pour structurer le contenu des pages web, tandis que le CSS permet de gérer l'apparence visuelle : couleurs, positionnement, polices, marges, et autres aspects esthétiques. En d'autres termes, le HTML constitue le "squelette" du site, alors que le CSS en assure le "design" et l'habillage visuel.

Un fichier HTML débute toujours par la déclaration <!DOCTYPE html>, qui indique au navigateur qu'il s'agit d'un document conforme au standard HTML5. Ensuite, tout le contenu est encadré entre les balises <html> et </html>. Ce contenu se divise en deux parties principales. La première, la balise <head>, regroupe les métadonnées de la page, telles que le titre affiché dans l'onglet du navigateur, ainsi que les liens vers des ressources externes comme des polices ou des feuilles de style CSS. Voici un exemple de contenu d'un <head> :

```
<head>
   <link href='https://fonts.googleapis.com/css?family=Playfair Display'
   rel='stylesheet'>
   <title> Projet PSE</title>
   <link rel="stylesheet" href="style.css">
</head>
```

La seconde partie, située entre les balises <body> et </body>, contient quant à elle tous les éléments visibles par l'utilisateur sur la page web. Par exemple, le début de notre page d'accueil s'écrit ainsi :

```
<body style="background-image: url('img/FondAcc.png');">
    Image de fond
   <div class="PageHeader">
       En-tete de la page
      <img id="logoAcc" src="img/logo.png">
      Logo du site
      <h1 id="titre1" style="color: rgb(0, 0, 0)">InsaNews</h1>
       Titre du site
   </div>
```

Le tableau suivant présente un résumé des balises HTML les plus couramment utilisées, avec une brève description de leur rôle dans la structure d'une page web.

Balise	Description
`<h1>` à `<h6>`	Titres de différents niveaux, `<h1>` étant le plus important.
`<p>`	Paragraphe de texte.
`<a>`	Lien hypertexte. Utilise l'attribut `href` pour la destination.
`<img>`	Image. Nécessite l'attribut `src` pour le fichier image.
`<div>`	Conteneur générique pour structurer la page.
`<input>`	Champ de saisie utilisateur dans un formulaire.
`<form>`	Conteneur pour les éléments de formulaire, utilisé pour envoyer des données.
`<button>`	Bouton cliquable, souvent utilisé pour soumettre un formulaire.

TABLE 2.1 – Résumé des balises HTML essentielles

Afin de concevoir notre page d'accueil et notre page de sélection, nous avons d'abord créé un schéma pour déterminer quels éléments intégrer et où les placer :



FIGURE 2.1 – Schéma de la page d'accueil

Nous avons d'abord intégré une image de fond en utilisant l'attribut style="background-image : url('img/FondAcc.png');" directement sur la balise <body>. Dans une division nommée "PageHeader", nous avons inséré le logo du site à l'aide de la balise <img>, suivi du titre principal encadré dans une balise <h1>, qui affiche le nom "InsaNews" en noir.

Nous avons ensuite ajouté une section d'introduction et de connexion, intégrée dans une balise , qui affiche un message de bienvenue destiné à attirer l'attention de l'utilisateur. Cette section est accompagnée de deux boutons : "Se connecter" et "Créer un compte". Ces boutons sont chacun encapsulés dans une balise <button> et reliés à une autre page HTML grâce à l'élément <a href="...">, qui permet de rediriger l'utilisateur vers la page correspondante lors d'un clic.

Au début du projet, nous avions prévu de permettre aux utilisateurs de créer un compte et de se connecter. Cependant, nous avons rapidement compris que ce n'était pas possible avec un site statique comme le nôtre, qui utilise uniquement HTML, CSS et JavaScript côté client. Pour gérer des comptes, il faut un site dynamique, avec un serveur et une base de données pour stocker les informations des utilisateurs, or nous n'avions pas les moyens de le mettre en place. Plutôt que de supprimer complètement cette partie de l'interface, nous avons décidé de garder les deux boutons "Se connecter" et "Créer un compte", mais de faire en sorte qu'ils redirigent tous les deux vers la page suivante, celle de la sélection des centres d'intérêt. Cela permet de conserver une navigation fluide sans avoir besoin d'un vrai système de connexion :

```
<a href="selection.html"> Lien vers la page de selection
   <button type="button" id="Login">Se Connecter</button> Bouton de
      connexion
   <button type="button" id="Signup">Creer un compte</button> Bouton
      creer un compte
</a>
```

Enfin, une image illustrative positionnée à droite de l'écran a été ajoutée à l'aide d'une balise <img>, afin de rendre l'interface plus visuelle et engageante. Nous obtenons donc le résultat suivant :



FIGURE 2.2 – Rendu final de la page d'accueil

2.1.2 Réalisation de la page de sélection

La page de sélection des centres d'intérêt a été développée dans le but de permettre à l'utilisateur de personnaliser les thématiques d'actualité qui l'intéressent avant d'accéder à la page principale du site. Cette interface est accompagnée d'un script en JavaScript, développés par d'autres membres du groupe, que nous détaillerons dans une section ultérieure.



FIGURE 2.3 – Schéma de la page de sélection

Pour structurer cette page de manière claire et esthétique, nous avons choisi de répartir les choix de thématiques en deux grandes sections, l'une située sur la première ligne et l'autre sur la deuxième ligne, chacune contenue dans une balise avec la classe checkboxes. La première section regroupe les quatre premiers centres d'intérêt (cinéma, histoire, politique, musique), tandis que la seconde présente les quatre suivants (écologie, faits divers, sport, jeux vidéo).

Chaque option proposée est représentée sous forme de bloc combinant une case à cocher, une image et un titre. Concrètement, chaque bloc contient une balise <input type="checkbox"> suivie d'un élément <label>. Ce label intègre une illustration associée à la thématique, insérée grâce à une balise <img>, ainsi qu'un titre visible, encadré par une balise <h3>. Le lien entre la case et son label est établi via l'attribut for du label, qui fait référence à l'identifiant (id) de l'input correspondant. Cela permet à l'utilisateur de cliquer non seulement sur la case, mais aussi directement sur l'image ou le texte pour effectuer la sélection :

```
<div class="cases"> Case a cocher pour le cinema
   <input type="checkbox" id="cb1" name="check1" />Type,identifiant et
      nom
   <label for="cb1">
      <img id="cineIMG" src="img/cinema.png">
      <h3>Cinema</h3>
   </label> Son contenu
</div>
```

La mise en forme visuelle de tous ces éléments a été assurée à l'aide du CSS. Nous avons par exemple utilisé les propriétés display : grid pour organiser les blocs de manière harmonieuse et leur attribuer des espaces réguliers. Les images ont été redimensionnées de manière homogène afin d'assurer une cohérence visuelle, et des marges ont été ajoutées autour des éléments pour favoriser l'aération de la page.

```
.checkboxes {
   display: grid; /* Grille pour faciliter l'organisation */
   margin-left: 80px; /* Marge a gauche */
   grid-template-columns: 1fr 1fr 1fr 1fr; /* Quatre colonnes */
   place-items: center; /* Centrer le contenu */
   justify-content: center; /* Justifier le contenu */
}
```

Enfin, afin d'apporter une touche de dynamisme à l'interface, nous avons intégré une animation simple sur le bouton "Valider". Lorsqu'un utilisateur survole ce bouton avec la souris, son apparence change légèrement, notamment au niveau de la couleur, grâce à une pseudo-classe CSS : hover. Ce type d'effet visuel renforce l'interactivité sans nécessiter de JavaScript. L'animation repose sur une transition fluide définie dans la feuille de style CSS, par exemple avec l'instruction transition : background-color 0.3s ease :

```
#valid{
    background-color:rgb(32, 30, 30); /* Couleur de fond */
    color: rgb(255, 255, 255); /* Couleur du texte */
    border: none; /* Pas de bordure */
    border-radius: 25px; /* Coins arrondis */
    padding: 15px 30px; /* Espacement interieur */
    font-size: 1.5em; /* Taille de la police */
    cursor: pointer; /* Curseur en forme de main */
    transition: background 0.3s; /* Transition pour l'arriere-plan */
}

#valid:hover {
    background: rgb(0, 162, 16); /* Couleur de fond quand on met le
        curseur */
    color: rgb(255, 255, 255); /* Couleur du texte quand on met le
        curseur */
}
```

Finalement, nous avons obtenu comme résultat cette page :

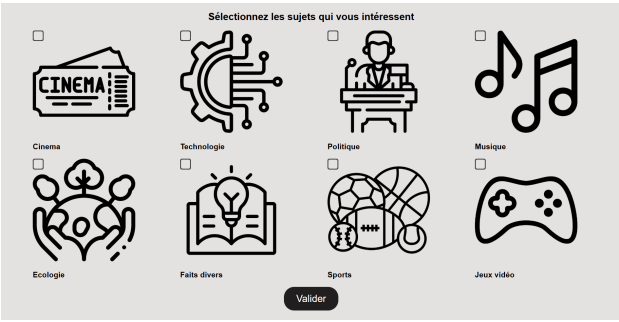


FIGURE 2.4 – Page de sélection



FIGURE 2.5 – Rendu après sélection

2.1.3 Réalisation de la page principale

Pour la page principale de notre site, nous avons cherché à créer une interface claire, agréable à utiliser et capable d'afficher des contenus dynamiques en fonction des choix de l'utilisateur. En suivant une structure à trois colonnes, nous avons séparé la page en différentes sections : la colonne de gauche, le centre, et la colonne de droite.

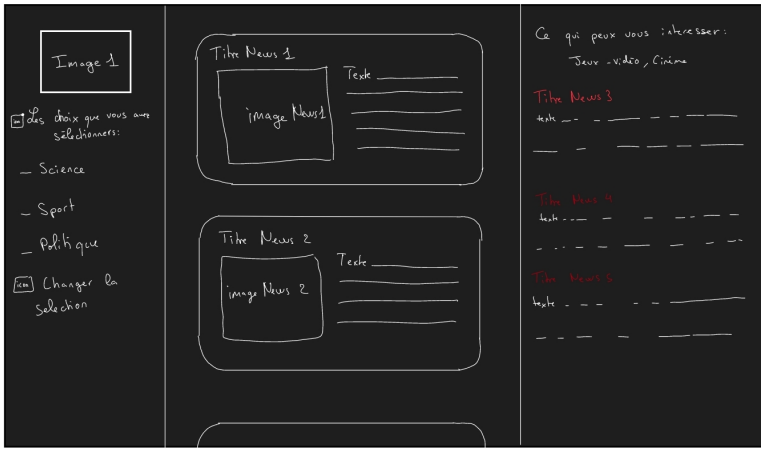


FIGURE 2.6 – Schéma de la page principale

Le contenu de la colonne de gauche est consacré à la présentation des choix sélectionnés par l'utilisateur. Nous avons utilisé une pour regrouper ces informations, avec la classe gauche. À l'intérieur de cette section, nous avons d'abord intégré un logo du site à l'aide de la balise <img> et appliqué une taille fixe grâce à l'attribut width. Cette image est accompagnée du texte descriptif placé sous des balises <p> qui est "Les choix que vous avez sélectionné". Ce texte sert à indiquer à l'utilisateur quelles catégories d'actualités il a sélectionné pour personnaliser son expérience. Pour afficher ces choix dynamiquement, nous avons inséré un élément avec l'id selected-themes, que nous avons ensuite rempli via JavaScript. Le lien pour changer les sélections est également placé dans cette section, avec un style d'<a> qui redirige l'utilisateur vers la page de sélection.

Au centre de la page, la section avec la classe centre affiche l'actualité principale. Cette zone est rendue interactive grâce à l'intégration d'une balise dont le contenu est alimenté en temps réel par JavaScript. Le script permet de récupérer les dernières actualités pertinentes via une API, comme GNews, et de les afficher directement dans la page en format dynamique.

La colonne de droite est conçue pour afficher des actualités supplémentaires susceptibles d'intéresser l'utilisateur. Elle reprend une structure similaire à celle de la colonne centrale, mais les articles y sont générés de manière aléatoire. Par défaut, sans l'intervention du JavaScript, le site affiche deux articles écrits en dur dans le code. Ces articles sont placés sous la balise et chacun est encapsulé dans un bloc . Chaque bloc comprend un titre, une brève description, ainsi qu'un style personnalisé (couleur, taille, etc.) défini via CSS. Exemple :

```
<div class="articles">
  <div class="apercu-actu">
    <div class="titre-actu"> Les plateformes americaines... </div>
    <p>Netflix, Amazon et Disney jouent le jeu, en... </p>
  </div>
</div>
```

Dans le fichier CSS, nous avons ajusté les dimensions et les espacements de chaque section pour une mise en page fluide. Par exemple, la propriété flex-direction : column utilisée pour les classes .gauche, .centre, et .droite permet d'organiser les éléments verticalement, tandis que les propriétés comme width : 15% et width : 60% définissent la taille relative de chaque colonne. De plus, des bordures blanches ont été ajoutées pour séparer visuellement les différentes sections de la page et apporter un contraste avec le fond sombre choisi pour

l'ensemble du site. Les images dans la section d'actualité sont également redimensionnées via CSS pour garantir une cohérence visuelle et un rendu homogène sur tous les appareils. Finalement, nous avons obtenu le résultat suivant :

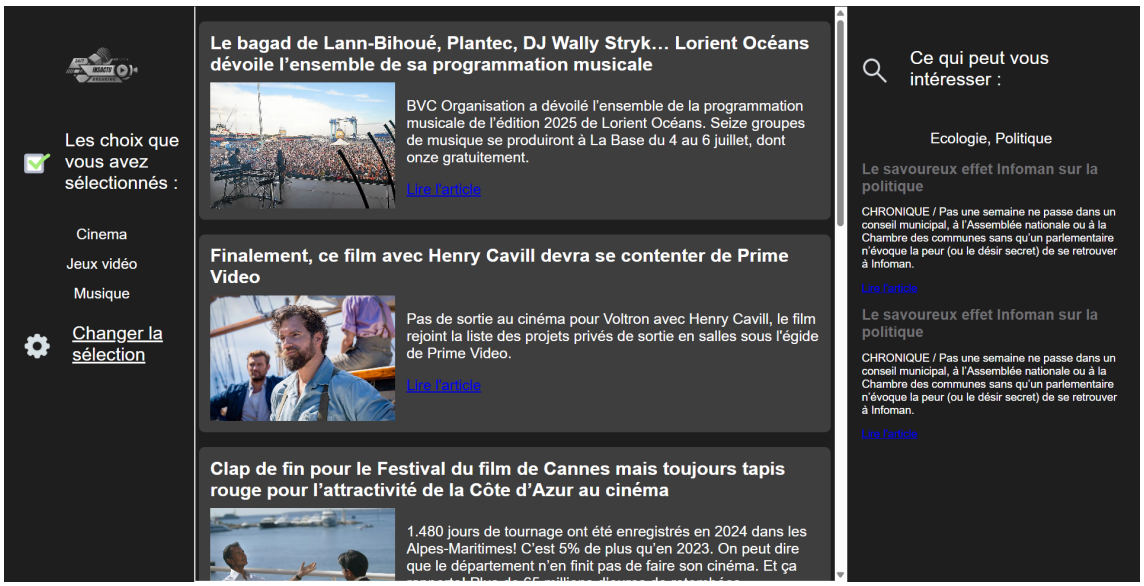


FIGURE 2.7 – Rendu finale de la page principale

2.2 Partie Logique

2.2.1 Recherche des APIs

Avant de commencer l'implémentation d'une quelconque API, il fallait réfléchir à laquelle choisir. Un gros travail de recherche a donc été fait afin d'identifier plusieurs APIs répondant à nos besoins :
- Une API ayant un accès gratuit.
- Une API généraliste, qui ne nécessite pas douze services pour une couverture large.
- Une API couvrant principalement l'actualité occidentale et des thématiques pertinentes pour un public étudiant.

Après un long travail de recherche et quelques essais, nous arrivons à identifier deux APIs qui répondent alors à nos besoins : MediaStack et GNews. Un nouveau travail de lecture de documentation a eu lieu pour mieux comprendre leur fonctionnement, afin de pouvoir mieux les intégrer ensemble. Une fois que l'ensemble des requêtes et formats de réponses étaient bien maîtrisés, nous avons pu commencer à travailler sur l'intégration de toutes ces APIs et de leur cohabitation sur notre site.

2.2.2 Utilisation des APIs

Le langage utilisé dans toute cette partie est le JavaScript. Dans un premier temps, il nous a paru fondamental de comprendre comment faire un appel à une API mais aussi comment fonctionne les données retournées. Pour cela, nous avons d'abord développé une première version d'une fonction fetchNews, qui nous permettait de nous concentrer sur l'aspect récupération des données, sans intégration dans une interface utilisateur particulière.

Cette première fonction nous a d'abord permis de comprendre comment utiliser le langage JavaScript et ses particularités, elle permettait également de vérifier que l'appel à l'API fonctionnait et d'identifier les propriétés des articles qui étaient retournées, à savoir : le titre, la description, l'image associée et l'URL de redirection vers l'article complet. Ce premier prototype en JavaScript nous a notamment permis de trier les différentes APIs que nous avions au préalable jugées comme potentiellement intéressantes pour notre projet.

Cette étape a mis en avant deux APIs qui nous paraissaient plus judicieuses à utiliser en raison du nombre de requêtes mises à disposition gratuitement et de leur accessibilité. Ces APIs que nous avons donc utilisées pour la suite de notre projet sont GNews et MediaStack, comme mentionné dans la partie précédente.

Expliquons en détail la première version de la fonction fetchNews que nous avons développée. Avant toute chose, il est nécessaire de définir comme constante l'URL de l'API que nous souhaitons utiliser ainsi que la clé API associée, comme montré ci-dessous.

```
function fetchNews() {
  const apiKey = 'f83f3ede3b6ee1437808418ef24b56f9';
  const url = `https://gnews.io/api/v4/search?q=example&lang=en&\
     country=us&max=10&apikey=${apiKey}`;
}
```

Il est important de noter ici que ce ne sont pas des guillemets (') qui sont utilisés autour de l'url mais des backticks (`) car ces derniers permettent l'interpolation de variable en JavaScript, c'est-à-dire que l'on peut utiliser dans l'URL la constante définie juste avant à

savoir la clé API. On remarque également que dans l'URL indiquée, on peut appliquer différents filtres. Dans l'exemple donné ci-dessus, on filtre en retournant uniquement les articles en anglais.

```
fetch(url)
    .then(response => response.json())
    .then(data => {
      const container = document.getElementById('news-container');
```

Ensuite, une fois l'URL définie, nous utilisons la fonction fetch(), qui est une fonction intégrée du langage JavaScript, pour envoyer une requête HTTP GET. L'utilisation de ".then" sert à exécuter le code lorsque la réponse de l'API a été reçue. Pour continuer, le paramètre "data" contient le résultat de la requête au format JSON, à savoir les différents articles avec leurs propriétés (titre, URL, description, image). Le "container" est ce qui va apparaître dans la page web. On va y insérer des éléments HTML générés dynamiquement en JavaScript, comme le montre le code suivant :

```
if (data.articles) {
      data.articles.forEach(article => {
        const articleDiv = document.createElement('div');
        articleDiv.innerHTML = `
          <h2>${article.title}</h2>
          <p>${article.description || 'Pas de description
             disponible.'}</p>
          ${article.urlToImage ? `<img src="${article.urlToImage}"
             alt="Image" style="max-width: 300px;">` : ''}
          <p><a href="${article.url}" target="_blank">Lire
             l'article</a></p>
          <hr>
        `;
        container.appendChild(articleDiv);
      });
    } else {
      container.innerText = "Aucun article trouve.";
    }
```

Cette portion de code commence par la condition "if (data.articles)" qui sert à vérifier que le paramètre "data" contient bien des articles, c'est-à-dire que le résultat de la requête n'est pas vide. S'il n'y a aucun article présent, il est affiché "Aucun article trouvé." sur la page web. Sinon une boucle "forEach" parcourt chacun d'eux. Pour chaque article, un bloc HTML est généré dynamiquement contenant le titre, la description (ou un message alternatif si elle est absente), une image (si disponible) et un lien vers l'article complet. Ce bloc est ensuite ajouté dans le "container" grâce à "appendChild", ce qui permet l'affichage sur la page web.

C'est grâce à cette fonction que nous avons pu tester les différentes APIs et observer ce que l'on pouvait en tirer.

2.2.3 Interface unifiée pour l'utilisation des APIs

Après avoir expérimenté et compris le fonctionnement de base d'une API grâce à notre première implémentation de la fonction fetchNews, nous avons souhaité structurer le projet autour d'un mécanisme plus robuste, capable de gérer plusieurs sources d'information de manière centralisée et cohérente. C'est dans cette optique que nous avons développé une nouvelle fonction : fetchArticlesForThemes(voir annexe).

Le rôle principal de cette fonction est de récupérer les articles correspondant aux thèmes sélectionnés par l'utilisateur, en interrogeant simultanément les trois APIs retenues pour le projet : GNews et MediaStack. Pour gérer efficacement ces requêtes multiples, la fonction utilise une structure JavaScript spécifique appelée promesse (ou Promise). Une promesse est un objet qui représente une opération asynchrone, c'est-à-dire une opération dont le résultat n'est pas immédiatement disponible.

Par exemple, lorsque nous lançons une requête HTTP via la fonction fetch(), celle-ci ne retourne pas directement les données demandées, mais plutôt une promesse indiquant que ces données seront fournies ultérieurement, une fois la requête terminée. L'utilisation des promesses permet ainsi à notre programme de continuer à fonctionner sans blocage, même pendant l'attente des réponses des APIs.

Concrètement, la fonction fetchArticlesForThemes, qui prend en entrée la liste des thèmes sélectionnés par l'utilisateur (selectedThemes), débute par la création d'un tableau nommé fetchPromises, destiné à stocker les promesses retournées par chacune des requêtes vers les APIs. Une boucle utilisant la méthode JavaScript .forEach() parcourt le tableau ['GNews', 'NewsApi', 'MediaStack'], et pour chaque API, appelle la fonction fetchNews(api, selectedThemes).

Cette dernière se charge de générer dynamiquement l'URL spécifique à chaque API, en s'appuyant sur deux objets clairement définis dans le script : apiUrls (qui contient les modèles d'URL adaptés à chaque API) et apiKeys (qui regroupe les clés nécessaires pour authentifier chaque requête). Une fois l'URL construite, fetchNews effectue une requête HTTP et retourne la promesse associée à cette opération.

Toutes les promesses ainsi générées sont alors regroupées dans le tableau fetchPromises. Nous utilisons ensuite la méthode JavaScript Promise.all(fetchPromises) pour attendre que toutes les promesses soient résolues simultanément. Ce mécanisme présente un avantage majeur : il permet de réaliser les trois requêtes vers les différentes APIs en parallèle plutôt que successivement, ce qui réduit considérablement le temps d'exécution.

Lorsque toutes les réponses sont disponibles, elles sont fusionnées en un unique tableau grâce à la méthode .flat(), puisque chaque API retourne ses résultats sous forme d'un tableau distinct. Le tableau obtenu subit ensuite un filtrage afin d'éliminer les éventuels doublons (voir 2.2.6 pour voir le code et avoir des détails). Pour assurer une diversité d'affichage, il est ensuite mélangé aléatoirement grâce à la fonction shuffleArray(). Enfin, le tableau final (allArticles) est transmis à la fonction genererArticles(allArticles), chargée de générer dynamiquement les blocs HTML nécessaires à l'affichage des actualités sur la page.

En complément des articles principaux correspondant aux thèmes choisis, la fonction fetchArticlesForThemes est également responsable de générer une section complémentaire appelée sidebar, afin de proposer à l'utilisateur des articles liés à des thèmes non sélectionnés initialement.
Pour cela, une fonction spécifique nommée `themesNonChoisieAleatoire()` est utilisée : elle choisit aléatoirement deux thèmes parmi ceux qui n'ont pas été cochés par l'utilisateur, si cela est possible (nous reviendrons sur cette fonction plus en détail après).

Une fois ces deux thèmes complémentaires sélectionnés, une nouvelle requête est effectuée exclusivement auprès de l'API GNews, via la fonction fetchNews('GNews',

sidebarThemes). La réponse obtenue est filtrée pour ne retenir que les articles valides (avec un titre non vide), puis le nombre d'articles affichés dans la sidebar est limité aux deux premiers résultats. Chaque article retenu est formaté en un objet simplifié ne conservant que les propriétés essentielles suivantes : titre (titre de l'article), contenu (description ou texte alternatif) etlien (URL directe vers l'article complet).

Enfin, ces articles sont transmis à la fonction updateSidebarWithTheme(sidebarThemes, sidebarArticles), qui se charge d'insérer dynamiquement les éléments correspondants dans la section latérale de la page, en indiquant clairement à l'utilisateur les deux thèmes proposés. Cette sidebar offre ainsi à l'utilisateur une ouverture sur des sujets supplémentaires qu'il n'aurait pas nécessairement consultés spontanément, enrichissant ainsi son expérience sur notre site.

2.2.4 Récupération des choix utilisateurs

Dans cette partie on va détailler un peu plus en détail le fonctionnement de la récupération et la communication des thèmes choisis entre nos différentes pages. On va commencer par la page de sélection des thèmes :

```
document.getElementById('valid').addEventListener('click',
    function(event) {
    event.preventDefault();

    const results =
       Array.from(document.querySelectorAll('input[type="checkbox"]:checked'))
    .map(checkbox => {

    const label = checkbox.nextElementSibling;
    const labelText = label.querySelector('h3').textContent;
    return labelText.trim();
});
```

La première ligne assigne les lignes de code qui suivent au bouton "Continuer" de la page. Ensuite la ligne qui exécute `preventDefault()` permet d'annuler le comportement par défaut du bouton, c'est-à-dire l'envoi d'un formulaire. On crée alors une constante results qui va contenir tous les checkboxes cochés de la page sous le type NodeList, propre à JavaScript, que l'on va passer dans un `Array.from()` pour obtenir un tableau d'éléments HTML.
Ensuite on attribue à label le nextElementSibling de notre checkbox (i.e l'élément HTML qui suit directement notre checkbox, soit le texte associé), et on récupère le titre (le <h3>) associée ce qui correspond au nom de la checkbox. Enfin la fonction trim permet d'enlever les éventuels espaces. L'opération de récupération des titres va renvoyer une chaîne de caractères pour chaque checkbox cochée, et la fonction `map()` va créer une liste de titres pour chacun de ces titres.

On obtient alors une variable results, qui est un tableau de chaîne de caractères, contenant tous les thèmes choisis. On peut alors l'exploiter pour envoyer ce tableau au code Javascript de la page suivante, deux choix s'offrent alors à nous :

Première solution, passer par les sessions storage, c'était notre idée de base : tous les navigateurs internet proposent un espace de stockage en local (sur la machine). Ces systèmes de stockage fonctionnent donc sur un système de dictionnaire (on associe à une

clé, une certaine valeur de notre choix), que l'on peut appeler sur n'importe quel autre page de notre document. Ce système était idéal et assez fiable, mais demandait beaucoup d'implémentation alors qu'un autre système plus simple et plus efficace existait.

La deuxième solution, plus pragmatique au vu de la faible quantité de données à transmettre, était de passer directement par l'URL du site via l'API HTTPS utilisé pour charger la page. En effet, l'URL constitue également une source de stockage très efficace pour stocker des données (elle est en réalité passée en paramètre de notre requête). Une fois la page chargée, on peut alors récupérer cette URL et lire ses informations sur la nouvelle page. Observons plus concrètement l'implémentation :

Côté page de sélection, on a ce simple code :

```
if(results.length === 0) {
   alert("Merci de selectionner au moins un sujet.");
}
else{
   // Envoi des resultats a la page de resultats
   window.location.href = "page_principale.html?results=" +
      encodeURIComponent(JSON.stringify(results));
}
```

Si aucune case n'a été cochée, un avertissement est imprimé dans la console, et rien ne se passe pour l'utilisateur. En revanche, si une case ou plus a été cochée, on va modifier l'URL, par la page suivante pour la charger, en passant des informations grâce aux informations après le "?" dans l'URL, via l'API HTTPS.
On va se concentrer sur les moyens d'encoder (i.e passer d'une structure de données à un autre type lisible de données pour le navigateur). Concrètement, `JSON.stringify` renvoie une version en chaîne de caractères de la structure de données. Par exemple, le tableau contenant les nombres 1, 2 et 3 sera renvoyé en "[1,2,3]". Ensuite `encodeURIComponent()` permet de convertir des caractères indésirables dans les URLs (comme "?" ou "-" par exemple) dans un format autorisé (souvent sous la forme %XX par exemple). Une fois ça on peut donc transmettre les données.

Côté affichage on retrouve alors cette fonction :

```
function getThemes() {
  let url = window.location.href;
  let params = url.split('?')[1];
  let results = params.split('=')[1];
  results = decodeURIComponent(results);
  results = JSON.parse(results);
  results.sort();
  return results;
}
```

Concrètement, cette fonction fait l'exact inverse de ce que l'on a fait précédemment, on va dans l'ordre récupérer le contenu dans l'URL de la page, et isoler la partie après le "=" qui contient uniquement les données que l'on souhaite récupérer.
Sur cette chaîne de caractère, on va appliquer la fonction `decodeURIComponent()` qui permet de générer les caractères interdit dans l'URL que l'on a transformé précédemment. Et `JSON.parse()` permet de transformer une chaine de caractères encodé par la bibliothèque dans le bon type de donnée, qu'on avait initialement fait.
Enfin on trie le tableau, de manière à être sûr que l'ordre des cases cochées ou la manière

dont elles sont envoyées, n'ait aucun impact sur la suite du code, de manière à avoir toujours le même ordre.

On va également détailler la fonction themesNonChoisieAleatoire() dont on a parlé précédemment, dont le code est disponible en annexe.

La première ligne récupère tous les thèmes à l'aide d'un dictionnaire déclaré en constante précédemment, (il répertorie l'id des checkboxs avec le nom du thème associé), on récupère ainsi toutes les valeurs de ce dictionnaire. De cette liste complète on retire les thèmes sélectionnés à l'aide d'un filtre (l'implémentation est similaire à celle de Python par exemple). A partir de là trois cas se distinguent : le premier est le cas où l'utilisateur a sélectionné tous les thèmes, celui où il a tout coché sauf un thème, et le dernier où il n'y a plus de choix.

Dans le premier cas, on prend deux thèmes déjà sélectionné, faute de choix. Pour cela on utilise un syntaxe Javascript un peu étrange, que l'on va détailler tout de suite. En effet, la fonction `sort()`, comme dans tous les langages, permet de trier un tableau d'éléments, là où ça devient intéressant, c'est qu'en Javascript, cette fonction peut prendre en paramètre une fonction de comparaison qui doit prendre deux paramètres (des éléments de la liste), et qui doit renvoyer un réel. De ce réel, la fonction `sort()` réagira de la manière suivante :

- Si le nombre renvoyé est 0, elle ne changera pas les éléments de place entre eux, mais les triera par rapport au reste (cas de deux mêmes éléments globalement).
- Si le nombre renvoyé est strictement inférieur à 0, elle triera les deux éléments de sorte que le premier soit avant le deuxième.
- Si le nombre renvoyé est strictement supérieur à 0, elle triera les deux éléments de sorte que le deuxième soit avant le premier.

Dans notre cas c'est assez pratique pour mélanger une liste aléatoirement : il suffit de donner une fonction de comparaison qui fait "n'importe quoi", et c'est exactement ce que l'on fait : on donne une fonction anonyme (i.e qui n'a pas de nom, on la déclare sur une ligne), qui ne prends aucune entrée et qui renvoie un réel aléatoire entre -0.5 et 1.5. La liste est alors mélangée selon les règles ci-dessus, on peut récupérer les deux premiers éléments et avoir nos choix aléatoires.
Note : pour cette partie on aurait très bien pu utiliser une structure similaire au deuxième cas, en générant deux numéros d'indice aléatoire, mais nous voulions varier un peu les syntaxes,de sorte que notre code présente aussi une certaine variété.

En parlant du deuxième cas justement, il est un peu plus simple, puisque dans ce cas, on va prendre l'unique thème non sélectionné disponible, et un des autres thèmes sélectionnés au hasard. (Petite précision : En JavaScript, Math().random() génère un réel aléatoire entre 0 et 1 inclus). Donc en prenant la partie entière du produit de ce nombre et de la taille de la liste, on retrouve bien un indice aléatoire. Le troisième cas est très analogue au premier, on prend simplement cette fois-ci le tableau de thèmes non-sélectionnés.

2.2.5 Intéractions avec le HTML

On va maintenant s'intéresser aux fonctions qui se chargent de modifier dynamiquement le contenu de la page HTML, directement depuis le script. On va traiter à chacune des fonctions qui traitent les divs de la page HTML, commençons par la partie à gauche :

```
function renderThemes(themeList, containerId) {
```

```
  const container = document.getElementById(containerId);
  container.innerHTML = "";


  themeList.forEach(theme => {
    const p = document.createElement("p");
    p.textContent = theme;
    container.appendChild(p);
  });
}
```

Cette fonction se charge simplement de charger les différents thèmes choisis dans la partie gauche de la page principale. Concrètement, la fonction récupère alors la de gauche contenant une liste de paragraphe <p> pour afficher la liste de thème. Notre fonction récupère cette , et pour chaque thème dans notre liste de thème, insère le paragraphe <p> correspondant dans le HTML.

Avant de passer à la suite, nous allons présenter un type centrale dans la suite du code, le type article :

```
titre: "Municipales 2026 : la Metropole du Grand Paris, enjeu de
   l'election dans la capitale",
image: "img/image_test_1.avif",
contenu: "Plusieurs candidats de gauche font de ce sujet un axe de leur
   campagne.",
lien: "details.html"
```

C'est une simple structure (comme on le voit dans l'exemple içi) qui stocke les différentes données essentielles à l'affichage des articles, elle est à la fois générée par les fonctions d'accès API, et utilisée par les fonctions d'affichages.

Parlons maintenant des trois fonctions d'affichages, qui sont plutôt similaires comme on va le voir. Commençons par la fonction qui génère les articles au centre, dont le code est disponible en annexe, sous la fonction genererArticle() :

Ce code récupère la div associée à la boîte centrale, une fois fait, elle y retire tout le contenu et va remplir selon les articles associés. Pour aller plus en détail, elle va créer une nouvelle div pour chaque article, et va lui attribuer lors du clic, la redirection vers le lien associé à l'article. La ligne suivante va s'occuper de générer le lien de l'image de secours. La ligne va d'abord récupérer le thème de l'article, ou utiliser la valeur "default" s'il est manquant, puis transformer ce thème en minuscules, supprimer les accents et remplacer les espaces par des underscores afin d'obtenir un nom de fichier valide (par exemple "Faits Divers" devient "faits_divers").

La fonction va alors construire dynamiquement le chemin de l'image de secours sous la forme 'img/faits_divers.png'. Ce qui a pour effet de charger une image localement selon le thème, dans le cas où l'article n'aurait pas d'image associée. Enfin on va ajouter le code HTML correspondant à un article "modulable" selon nos données, celui-ci est directement pris du code fait par Maïssa et Mohammed, et adapté pour un affichage modulaire. Enfin on finira par ajouter la div nouvellement généré au conteneur centrale.

Enfin, voyons l'affichage à droite, qui regroupe les deux types d'opérations que nous venons de voir, le code de la fonction updateSidebarWithTheme() est disponible en annexe.

Cette fonction commence comme les deux autres, en récupérant et en vidant la section de droite. On va ensuite créer deux div : la première servira de classe **choix_d** servira à afficher

les noms des deux thèmes non sélectionnés en générant deux paragraphes. L'autre div se chargera de générer deux articles dans le HTML, en faisant à nouveau un HTML formaté, qui sera généré avec les données de nos articles. Le CSS déjà implémenté dans la partie statique, se chargera de la mise en page.

2.2.6 Améliorations

Lorsque l'utilisateur sélectionne plus de deux catégories, les articles s'affichent les uns à la suite des autres par thème : d'abord tous les articles du premier thème, puis ceux du deuxième, et ainsi de suite. Une amélioration intéressante consisterait à mélanger les articles issus de chaque thème pour offrir une expérience de lecture plus fluide et variée. Tout se jouait autour de la fonction fetchArticlesForThemes. Le problème venait du fait que, dès qu'une requête API était effectuée pour un thème, les articles de ce thème étaient immédiatement affichés, sans attendre les autres. La solution a été d'attendre que toutes les requêtes, pour chaque API et chaque thème, soient envoyées et résolues, de stocker les résultats dans un tableau commun, de les mélanger, puis de les afficher ensemble.Le détail du code de la fonction fetchArticlesForThemes est disponible en annexe.

La fonction fetchArticlesForThemes a pour objectif de récupérer et traiter des articles d'actualité en fonction de thèmes sélectionnés. Elle commence par créer un tableau vide nommé fetchPromises destiné à stocker des promesses de requêtes. Ensuite, elle parcourt une liste de trois sources d'actualités : 'GNews', 'NewsApi', et 'MediaStack', et pour chacune d'elles, elle appelle la fonction fetchNews en lui passant l'API et les thèmes sélectionnés. Cette fonction renvoie une promesse, ajoutée au tableau.

Une fois toutes les requêtes lancées, Promise.all attend que l'ensemble des promesses soit résolu. Les résultats sont alors récupérés sous forme d'un tableau de tableaux d'articles. Ces résultats sont ensuite aplatis en un seul tableau avec flat(). On filtre ensuite ce tableau pour éliminer les doublons : on convertit les titres en minuscules et on enlève les espaces superflus pour normaliser les comparaisons. Un ensemble (Set) nommé seenTitles est utilisé pour mémoriser les titres déjà rencontrés, ce qui permet d'ignorer les articles aux titres identiques.

Après ce filtrage, les articles sont mélangés aléatoirement à l'aide d'une fonction shuffleArray, afin de ne pas afficher les articles toujours dans le même ordre.

Le code de la fonction shuffleArray qui permet de mélanger les articles, une fois les doublons supprimés, est la suivante :

```
function shuffleArray(array) {
  for (let i = array.length - 1; i > 0; i--) {
    const j = Math.floor(Math.random() * (i + 1));
    [array[i], array[j]] = [array[j], array[i]];
  }
  return array;
}
```

La fonction shuffleArray permet de mélanger aléatoirement les éléments d'un tableau en utilisant l'algorithme de Fisher-Yates. Elle parcourt le tableau à l'envers, en partant du dernier élément jusqu'au second, et à chaque étape, elle sélectionne un index aléatoire entre 0 et l'index courant. Les deux éléments situés à ces positions sont ensuite échangés à l'aide d'une syntaxe de déstructuration. Ce processus garantit un mélange équitable et efficace

des éléments. Une fois toutes les permutations effectuées, le tableau mélangé est retourné.

Une autre amélioration que nous avons apportée est la suppression des doublons parmi les articles récupérés. En effet, l'utilisation simultanée de plusieurs sources augmente le risque de recevoir plusieurs fois le même article, problème que nous avions régulièrement rencontré dans les premières versions de notre projet. Pour traiter efficacement ce problème, nous avons initialement envisagé d'utiliser l'URL de chaque article pour identifier les doublons, si deux articles avaient la même URL, nous supprimions automatiquement l'un des deux. Cependant, cette approche présentait un défaut majeur : certaines APIs pouvaient retourner exactement le même article, mais avec des URLs légèrement différentes, ce qui ne permettait pas d'identifier correctement tous les doublons.

```
const seenTitles = new Set();
   let allArticles = results.flat().filter(article => {
     const title = (article.title || article.titre ||
        '').trim().toLowerCase();
     if (!title || seenTitles.has(title)) return false;
     seenTitles.add(title);
     return true;
```

Pour pallier ce problème, nous avons opté pour une méthode fondée sur les titres des articles. Nous avons donc mis en place un mécanisme simple dans la fonction fetchArticlesForThemes mais performant basé sur une structure JavaScript appelée Set, que nous avons nommé seenTitles. Cette méthode consiste à parcourir le tableau d'articles fusionnés grâce à la fonction .filter(). Pour chaque article, nous extrayons le titre en utilisant la syntaxe (article.title || article.titre || ''), puis nous le normalisons en le mettant entièrement en minuscules et en supprimant les éventuels espaces inutiles à l'aide de la méthode .trim().

Cette normalisation permet d'éviter les erreurs dues à des différences mineures de formatage. Le principe du filtrage est simple : si un titre est vide ou déjà enregistré dans seenTitles, l'article associé est ignoré. En revanche, si le titre n'a jamais été vu auparavant, il est ajouté à seenTitles, et l'article correspondant est conservé.

Ce mécanisme garantit ainsi que l'utilisateur ne verra jamais deux articles identiques, améliorant considérablement la qualité et la clarté de l'interface utilisateur.
Pour une efficacité encore meilleure, il serait envisageable de combiner plusieurs critères simultanément, comme l'URL, le titre, voire même la description de l'article, afin d'obtenir une détection parfaite des doublons.

Une dernière amélioration a été réalisé au niveau de la page principale et de la page de sélection. En effet, lors de notre oral d'entraînement à la dernière séance, l'autre groupe nous a fait remarquer à juste titre que nous n'affichions rien si l'utilisateur n'avait rien coché sur la page de sélection, ou si nous n'arrivions pas à charger les articles sur la page principale. Une petite correction a été faite en ce sens pour afficher une petite alert() JavaScript, qui permet de prévenir l'utilisateur des différents problèmes qui peuvent arriver ci-dessus, et de charger des articles par défaut pour la deuxième situation ...

Conclusion et perspectives

Ce projet nous a permis de découvrir (ou d'approfondir pour certains) les notions d'échanges client-serveur à travers l'utilisation d'APIs, dans notre cas focalisées sur les actualités, dans le cadre de la création d'un site web interactif. Nous avons également pu, grâce à ce projet, mettre en pratique certains langages informatiques, à savoir HTML, CSS et JavaScript, mais également de concepts plus avancés comme l'utilisation des promesses et du format JSON par exemple. Pour la plupart d'entre nous, cela constituait une nouveauté et donc nous avons eu l'occasion de découvrir ces notions. Au-delà de cet aspect technique, le projet scientifique encadré nous a appris à mieux travailler en équipe, à nous organiser de manière plus efficace et à nous entraider pour surmonter des problèmes, comme les bugs ou les limites dues aux APIs que nous utilisions. Nous avons tous pu acquérir de nouvelles compétences techniques tout en développant notre esprit collaboratif et en améliorant notre capacité d'organisation.

En ce qui concerne les objectifs que nous nous étions fixés, à savoir proposer des actualités personnalisées en temps réel à l'aide des APIs, nous les avons atteints. Cependant, plusieurs pistes d'améliorations s'offrent désormais à nous. En effet, le passage à un site web dynamique avec une base de données rendrait possible l'implémentation d'un système de comptes. Cela pourrait conduire au développement du site avec pour objectif une sauvegarde des préférences de l'utilisateur, incluant une possibilité de mettre des articles en favori par exemple. Enfin, à plus long terme, ce projet pourrait être la source d'une plateforme plus aboutie, capable de proposer du contenu de manière encore plus personnalisée, en tenant compte des centres d'intérêt et des habitudes de lecture de l'utilisateur.

Pour résumer, ce projet a été un exercice de réflexion intéressant et il constitue une expérience enrichissante, tant sur le plan technique, avec la découverte de nouveaux langages informatiques et de nouveaux concepts, que sur le plan humain, avec cet esprit collaboratif dont nous avons fait preuve.

Annexe A

Notre usage de l'IA dans ce projet

Durant le projet, nous avons eu plusieurs fois recours à l'intelligence artificielle en parallèle de la documentation officielle, de différents sites web et de la console de notre éditeur de code pour nous accompagner et nous guider. En particulier, comme nous ne connaissions pas bien les langages de programmation utilisés (notamment JavaScript), l'IA nous a permis de mieux comprendre leur fonctionnement. Elle nous a expliqué comment utiliser les APIs, comment structurer notre code au sens global, et nous a aidés à saisir certaines notions techniques parfois complexes pour des débutants, comme les promesses ou les fonctions asynchrones.

Elle nous a aussi donné des conseils sur les bonnes pratiques à adopter, les fonctions disponibles en JavaScript, ou les syntaxes possibles selon les cas. L'IA a donc joué un rôle de soutien, notamment lorsque nous rencontrions des erreurs où elle nous aidait à les localiser, à comprendre d'où venait le problème si on n'y arrivions pas à l'aide de la console seul, et à trouver des solutions. Elle ne nous donnait pas la réponse toute faite, mais nous orientait avec des explications ou des indices, ce qui nous permettait de nous corriger nous-mêmes.

En résumé, l'Intelligence Artificielle, nous servait parfois plus à nous mettre sur la voie, que de nous donner le chemin directement. Tout code qu'elle pouvait nous donner a été longuement testé, modifié et utilisé de manière à bien comprendre toutes les spécificités du code qui pouvait être renvoyé, et d'être sûr de la qualité de la fonction en retour.

Annexe B

Liens de documentation

https://pse-2025-e6a99b.pages.insa-rouen.fr/ Lien vers le site.

https://gitlab.insa-rouen.fr/beraudrobert/pse-public/ Lien vers le git.

https://www.cnil.fr/fr/definition/interface-de-programmation-dapplication-api
Consulté le 5 février 2025

https://api.gouv.fr/guides/api-definition
Consulté le 7 février 2025

https://www.mulesoft.com/fr/api/what-is-an-api
Consulté le 10 février 2025

https://ibracilinks.com/blog/quelle-est-la-difference-entre-un-site-web-statique-et-un-site-web-dynamique
Consulté le 22 avril 2025

https://datascientest.com/front-end-vs-back-end
Consulté le 27 avril 2025

https://developer.mozilla.org/fr/docs/Web/API/URL
Consulté le 10 mars 2025

MDN - Array.sort()
Consulté le 12 avril 2025

https://developer.mozilla.org/fr/docs/Web/CSS
Consulté le 10 mars 2025

https://developer.mozilla.org/fr/docs/Web/HTML
Consulté le 10 février 2025

Cours W3Schools sur les checkbox
Consulté le 15 février 2025

https://css-tricks.com/snippets/css/a-guide-to-flexbox/
Consulté le 25 février 2025

Annexe C

Code Supplémentaire

Fonction themesNonChoisieAleatoire()

```
function themesNonChoisieAleatoire(selectedThemes) {
  const allThemes = Object.values(themeMap);
  const unselected = allThemes.filter(theme =>
     !selectedThemes.includes(theme));

  // Tout choisi
  if (unselected.length === 0) {
    const shuffled = allThemes.sort(() => 0.5 - Math.random());
    return shuffled.slice(0, 2);
  }

  // Seulement 1 de libre
  if (unselected.length === 1) {
    const other = selectedThemes[Math.floor(Math.random() *
       selectedThemes.length)];
    return [unselected[0], other];
  }

  // Plus de 2 libre
  const shuffled = unselected.sort(() => 0.5 - Math.random());
  return shuffled.slice(0, 2);
}
```

Fonction genererArticles()

```
function genererArticles(articles) {
  const container = document.getElementById("actu-box");
  container.innerHTML = "";


  articles.forEach(article => {
    const div = document.createElement("div");
    div.className = "boite-actu";
    div.onclick = () => location.href = article.lien || '#';


    const theme = article.theme || 'default';
    const themeClean =
       theme.toLowerCase().normalize("NFD").replace(/[\u0300-\u036f]/g,
       "").replace(/ /g, "_");
    const fallbackImage = `img/${themeClean}.png`;


    div.innerHTML = `
      <div class="titre-actu">${article.titre || article.title}</div>
      <div class="contenu-actu">
       <img src="${article.image || article.urlToImage}"
            onerror="this.onerror=null;this.src='${fallbackImage}'"
            alt="Image Actu">
       <div class="texte-actu">
         <p>${article.contenu || article.description || 'Pas de
            description disponible.'}</p>
         <a href="${article.url}" target="_blank">Lire l'article</a>
            <!-- Lien vers l'article -->
       </div>
      </div>
    `;
    container.appendChild(div);
  });
}
```

Fonction updateSidebarWithTheme()

```
function updateSidebarWithTheme(theme_name, articles) {
  const sidebar = document.querySelector(".choix_d");
  sidebar.innerHTML = ""; // Nettoie la sidebar
  // Titre de section
  const titreSection = document.createElement("div");
  titreSection.className = "choix_d";
  titreSection.innerHTML = `<p>${theme_name[0]}, ${theme_name[1]}</p>`;
  sidebar.appendChild(titreSection);
  const sidebar_2 = document.querySelector(".articles");
  sidebar_2.innerHTML = ""; // Nettoie la sidebar
  articles.forEach(article => {
    // Contenu de l'article
    const apercuActu = document.createElement("div");
    apercuActu.className = "apercu-actu";
    apercuActu.innerHTML = `
      <div class="titre-actu">${article.titre}</div>
      <div class="contenu_side-actu">
        <div class="texte-actu">
          <p>${article.contenu || article.description || "Aucune
             description disponible."}</p>
          <a href="${article.lien}" target="_blank">Lire l'article</a>
        </div>
      </div>
    `;
    sidebar_2.appendChild(apercuActu);
    });
}
```

Fonction fetchArticlesForThemes()

```
function fetchArticlesForThemes(selectedThemes) {
  let fetchPromises = [];

    ['GNews', 'NewsApi', 'MediaStack'].forEach(api => {
      fetchPromises.push(fetchNews(api, selectedThemes));
    }); //On recupere tous les articles

  Promise.all(fetchPromises).then(results => {
    const seenTitles = new Set();
    let allArticles = results.flat().filter(article => {
      const title = (article.title || article.titre ||
          '').trim().toLowerCase();
      if (!title || seenTitles.has(title)) return false;
      seenTitles.add(title);
      return true;
    }); //On enleve les doublons

    allArticles = shuffleArray(allArticles); //On melange les articles

    genererArticles(allArticles); //On procede au rendu
    ...
}
```